YOLOv13 cho định vị miếng dán đối kháng vật lý: Benchmark có kiểm soát và phân tích ablation trên AdvT-shirt-1K

Tác giả: Phạm Tuấn Anh, Mai Quốc Bảo — Trường Đại học Công nghệ TP.HCM (HUTECH)

Giảng viên hướng dẫn: Đinh Phương Nam

Ngày báo cáo: 05 tháng 10 năm 2026

Phiên bản: Bản thảo nghiên cứu 2.0; số liệu truy xuất từ benchmark và ablation đã lưu trong dự án

Abstract (English)

Physical adversarial patches can alter object detector outputs in real scenes, while patch localization is only one part of a broader robustness problem. We evaluate YOLOv13-N/S/L against YOLOv8, YOLOv10, YOLO11 and YOLOv12 on patch-only localization derived from AdvT-shirt-1K. Since labeled splits for the original project are empty, this external dataset is treated strictly as a proxy. Its local mirror contains 1,123 images and 1,548 patch boxes; a deterministic image-level 70/15/15 split contains 785/169/169 images. Fifteen models were fine-tuned with a shared 416-pixel, 50-epoch maximum protocol. YOLOv13-S reached 83.96% test mAP@0.5:0.95 and 99.45% mAP@0.5, a small advantage over YOLOv12-S in raw run metrics, with 0.25 ms higher GPU latency per image. YOLOv13-N and -L did not outperform validation-selected same-scale baselines. A single-seed YOLOv13-N ablation found a modest validation mAP decrease after removing HyperACE; removing FullPAD or replacing DS-C3k2 increased mAP but also changed latency or compute. These descriptive results do not establish causal or generalizable architectural gains, adversarial robustness, or mitigation of NMS denial-of-service. We report the full benchmark, provenance and limitations to support controlled follow-up on the original project dataset.

Keywords: object detection; physical adversarial patch; patch localization; YOLOv13; hypergraph; AdvT-shirt-1K.

Tóm tắt tiếng Việt

Miếng dán đối kháng in có thể làm suy giảm tính toàn vẹn của bộ phát hiện vật thể khi đưa mô hình từ môi trường số ra thế giới thực. Nghiên cứu này đánh giá YOLOv13 trên tác vụ định vị hộp miếng dán, đồng thời đối sánh có kiểm soát với YOLOv8, YOLOv10, YOLO11 và YOLOv12. Các split có nhãn của bộ dữ liệu NCKH gốc hiện chưa có ảnh và nhãn, vì vậy AdvT-shirt-1K được dùng như tập proxy bên ngoài. Bản mirror có 1.123 ảnh và 1.548 hộp miếng dán; split xác định theo seed 42 gồm 785/169/169 ảnh train/validation/test. Mười lăm mô hình Nano/Small/Large được fine-tune với đầu vào 416×416 và lịch huấn luyện đồng nhất. YOLOv13s đạt 83,96% mAP@0.5:0.95 và 99,45% mAP@0.5 trên test, với 9,55 triệu tham số, 9,15 GFLOPs, độ trễ GPU 3,16 ms/ảnh và CPU 38,95 ms/ảnh. YOLOv12s đạt 83,81% và 99,41%, với độ trễ GPU 2,91 ms/ảnh; đây là cấu hình có mAP@0.5:0.95 validation cao nhất trong 15 cấu hình và do đó được chọn báo cáo chính. YOLOv13s cao hơn YOLOv12s 0,16 điểm phần trăm mAP@0.5:0.95 trên test nhưng tăng 0,25 ms độ trễ GPU. Kết quả chỉ chứng minh khả năng định vị trên proxy, không trực tiếp chứng minh khả năng chống tấn công vật lý, quá tải NMS hay hiệu quả trên dữ liệu đề tài gốc.

Từ khóa: phát hiện vật thể; miếng dán đối kháng vật lý; định vị patch; YOLOv13; AdvT-shirt-1K; robustness.

> ***Đối chiếu nguồn số liệu:*** *cặp 83,81% mAP@0.5:0.95 và 99,41% mAP@0.5 thuộc YOLOv12s, không phải YOLOv13s. YOLOv13s có kết quả tương ứng là 83,96% và 99,45% trong benchmark CSV.*

1. Giới thiệu và mục tiêu nghiên cứu

Miếng dán đối kháng vật lý được in và đặt trong cảnh thật để gây nhầm lẫn cho bộ phát hiện. Đây là rủi ro đối với tính toàn vẹn của nhận thức thị giác trong phương tiện tự hành và thiết bị biên. Định vị vùng patch có thể hỗ trợ hệ thống nhận diện khu vực nghi vấn và kích hoạt kiểm tra bổ sung. Tuy nhiên, phát hiện được hộp patch không đồng nghĩa mô hình đã chống được tấn công hoặc ngăn được suy giảm dịch vụ. [1], [2]

YOLOv13 đề xuất HyperACE để khai thác tương quan đặc trưng bậc cao giữa nhiều vị trí và tỉ lệ; FullPAD phân phối đặc trưng đã tăng cường xuyên suốt pipeline; DS-C3k2 dùng các khối tích chập tách biệt theo chiều sâu nhằm giảm chi phí tính toán. Đây là giả thuyết kiến trúc từ công trình YOLOv13. Hiệu quả trên AdvT-shirt-1K phải được xác định bằng phép đo tại chỗ, không suy diễn trực tiếp từ kết quả COCO. [30]

Nghiên cứu trả lời ba câu hỏi:

- **RQ1 — Chất lượng định vị:** YOLOv13 định vị hộp miếng dán ra sao trên split AdvT-shirt-1K được chia lại?
- **RQ2 — Chi phí suy luận:** độ chính xác đi kèm độ trễ, FPS, tham số và FLOPs nào so với baseline cùng quy mô?
- **RQ3 — Mức độ phòng thủ:** các phép đo hiện có có chứng minh khả năng chống tấn công vật lý hoặc DoS/NMS-overload trong bài toán NCKH gốc không?

2. Tổng quan nghiên cứu

2.1. Tấn công đối kháng vật lý và suy giảm độ trễ

Nghiên cứu đối kháng bắt đầu từ nhiễu có chủ đích trên mô hình phân loại [15–17], rồi mở rộng sang miếng dán có thể in và hiện diện trong cảnh thật [1, 2, 26–28]. Các công trình về Sponge Examples, Daedalus, Phantom Sponges, Overload, SlowTrack và tấn công độ trễ gần đây khảo sát cách tải ứng viên và hậu xử lý có thể tác động độ trễ hoặc tài nguyên [3–9]. Dòng nghiên cứu Sponge Poisoning và SkipSponge tập trung vào tiêu thụ năng lượng/độ trễ qua cấu trúc mạng hoặc trọng số [24, 25]. Thuật toán tiến hóa và tìm kiếm hộp đen cung cấp cách tối ưu hóa đầu vào khi gradient không sẵn có [22, 23]; đây cũng là nền tảng liên quan đến hướng tối ưu miếng dán trong báo cáo NCKH trước. Tuy nhiên, các mục tiêu đó khác với tác vụ hiện tại: định vị hộp patch không đo attack success, số ứng viên trước NMS hay deadline của luồng camera.

Các nghiên cứu phòng thủ đối với patch tự nhiên và vật lý gồm phương pháp kháng patch trên phân loại, phát hiện patch, đặc trưng phân biệt và giải thích mô hình [29, 33–35]. Chúng đặt ra phép kiểm chứng rộng hơn như patch chưa thấy, ảnh sạch, độ trễ của bước phòng thủ và khả năng giữ chất lượng phát hiện. Ở đây, các nghiên cứu đó xác định khoảng cách giữa "nhận diện vùng patch" và "phòng thủ có hiệu lực"; chúng không được dùng làm baseline định lượng vì nhiệm vụ, dữ liệu và giao thức khác nhau.

2.2. Detector một giai đoạn và cơ sở YOLOv13

Các detector một giai đoạn như YOLO và SSD hướng tới suy luận trực tiếp, trong khi Faster R-CNN dùng đề xuất vùng; focal loss xử lý mất cân bằng lớp và DETR đưa ra dự đoán theo tập bằng transformer [11, 18–21]. Tích chập tách biệt theo chiều sâu là một hướng giảm phép tính [12]. COCO là benchmark phát hiện tổng quát [14], còn tài liệu COCO128 và YOLOv8 cung cấp bối cảnh về định dạng dữ liệu và phần mềm cho pipeline [10, 13]. YOLOv10 nghiên cứu huấn luyện NMS-free và thiết kế hiệu quả đầu-cuối [39]; YOLO11 được duy trì như một dòng phần mềm Ultralytics và tài liệu dự án nói rõ chưa có bài báo học thuật chính thức [40]; YOLOv12 khai thác attention cho detector thời gian thực [41]. Các biến thể có kiến trúc, pretrained weights và lịch sử tối ưu khác nhau, nên protocol fine-tune chia sẻ chỉ kiểm soát một phần khác biệt.

YOLOv13 dùng HyperACE để xây dựng tương quan hypergraph thích nghi bậc cao giữa các vị trí/scale, FullPAD để phân phối đặc trưng qua pipeline và DS-C3k2 để thay một số khối tích chập bằng thiết kế tách biệt. Hypergraph biểu diễn quan hệ nhiều-đến-nhiều vượt liên kết từng cặp [31]. Bài báo YOLOv13 báo cáo kết quả COCO và

ablation; mã iMoonLab là implementation dùng trong nghiên cứu [30, 38]. Bằng chứng COCO là động cơ khảo sát, không dự đoán thứ hạng trên tập patch khác miền.

2.3. Khoảng trống và câu hỏi nghiên cứu

AdvT-shirt-1K được công bố cùng nghiên cứu BadPatch, với ảnh vật lý của áo có thiết kế đối kháng và hộp người/patch; mirror dùng trong pipeline có giấy phép Apache-2.0 theo thẻ dữ liệu [32, 36, 37]. Nghiên cứu này khảo sát riêng lớp patch, tạo split dẫn xuất và fine-tune các họ YOLO theo cấu hình chia sẻ. Câu hỏi trọng tâm là: (RQ1) YOLOv13 có tăng chất lượng định vị patch trên split này không; (RQ2) lợi ích đó đổi lấy độ trễ và chi phí tính toán nào; (RQ3) dữ liệu hiện có có đủ để kết luận về phòng thủ vật lý hoặc DoS/NMS không? Câu trả lời được giới hạn vào proxy và protocol đã đo.

3. Thiết lập thực nghiệm

3.1. Dữ liệu và nhãn

AdvT-shirt-1K gồm ảnh áo có miếng dán trong bối cảnh vật lý, với nhãn hộp cho người và patch. Nghiên cứu này chỉ dùng nhãn patch và một lớp duy nhất. Bản mirror hiện có 1.123 ảnh, thấp hơn con số 1.131 ảnh mà nguồn tác giả nêu; chênh lệch tám ảnh chưa được giải quyết và được xem là hạn chế về tính toàn vẹn nguồn dữ liệu. [32, 36, 37]

Nguồn mirror không cung cấp split phù hợp cho quy trình này. Nhóm tạo split xác định 70/15/15 theo seed 42, phân tầng trong năm thư mục nguồn: train 785 ảnh/1.106 hộp; validation 169 ảnh/220 hộp; test 169 ảnh/222 hộp. Tổng cộng 1.548 hộp. Split được tạo ở mức ảnh, không theo người mặc, thiết kế patch hoặc phiên chụp; vì vậy ảnh tương quan có thể xuất hiện ở nhiều split. Hai nhãn lệch biên 5×10^{-7} do làm tròn đã được chặn về [0,1] trong bản split dẫn xuất.

Ảnh gốc rộng 856–2056 pixel và cao 1044–1598 pixel; mọi mô hình nhận đầu vào 416×416. Dataset YAML proxy là dataset/adv_tshirt_patch_detection.yaml. Cấu hình chuẩn dataset/data.yaml dùng path: ./dataset; các split train/validation/test của đề tài hiện có 0 ảnh và 0 nhãn, tên lớp object vẫn là placeholder và không được dùng để huấn luyện.

3.2. Phần cứng và phần mềm

Thành phần	Cấu hình
CPU	Intel Core i5-14600KF
GPU	NVIDIA GeForce RTX 5060, 7,96 GiB VRAM
RAM	31,79 GiB
CUDA	Runtime 12.8
Python	3.14.7
PyTorch	2.11.0+cu128
Ultralytics	8.3.63
Môi trường	project_yolov13/.venv
Cấu hình YOLO	project_yolov13/.config
Dữ liệu, runs, weights	project_yolov13/dataset, runs, weights

Các đường dẫn dữ liệu, cache tải dữ liệu, cấu hình YOLO, checkpoint, log và kết quả nằm dưới thư mục dự án. Cây dataset được kiểm tra không có symlink/junction. Không có phép đo trên Jetson hay thiết bị edge chuyên dụng.

3.3. Cấu hình huấn luyện

Siêu tham số	Giá trị
Kích thước đầu vào	416×416

Siêu tham số	Giá trị
Batch size	4
Epoch tối đa / patience	50 / 10
Optimizer	SGD
Learning rate ban đầu	0,01
Scheduler	tuyến tính, lr cuối bằng 0,01 lần lr đầu
Warm-up	2 epoch
Weight decay	0,0005
Seed / deterministic	42 / bật
HSV	h=0,015; s=0,7; v=0,4
Hình học	translate=0,1; scale=0,5; horizontal flip=0,5
Mosaic	1,0
MixUp / CopyPaste	0 / 0

Mười lăm cấu hình dùng cùng split, seed, pretrained initialization tương ứng, augmentation và lịch huấn luyện. Mỗi cấu hình chỉ chạy một seed nên chưa có ước lượng phương sai hoặc kiểm định ý nghĩa thống kê.

3.4. Quy trình chọn mô hình và đo lường

Validation được dùng chọn mô hình/checkpoint; test chỉ dùng báo cáo cuối. YOLOv12s đạt validation mAP@0.5:0.95 cao nhất toàn bộ 15 cấu hình (84,78%), và đạt 83,81% mAP@0.5:0.95 cùng 99,41% mAP@0.5 trên test. Trong so sánh cùng scale, baseline được chọn là model có validation mAP@0.5:0.95 cao nhất ở scale đó.

Latency là thời gian suy luận trung bình mỗi ảnh theo Ultralytics validator tại batch 4, trên CPU/GPU; không gồm thời gian nạp mô hình. FPS* được tính bằng 1000/latency_ms, là nghịch đảo latency bình quân trên ảnh, không phải phép đo throughput video độc lập. Params và FLOPs do profiler Ultralytics đo ở 416 pixel. Các phép đo phụ thuộc thiết bị và runtime, không so trực tiếp với TensorRT FP16 trên Tesla T4.

4. Kết quả và benchmark

4.1. So sánh trên test split

Bảng 1. Mười lăm mô hình fine-tune trên test split AdvT-shirt-1K dẫn xuất. P/R/mAP tính theo phần trăm; latency là ms/ảnh ở batch 4; FPS* suy ra từ latency.

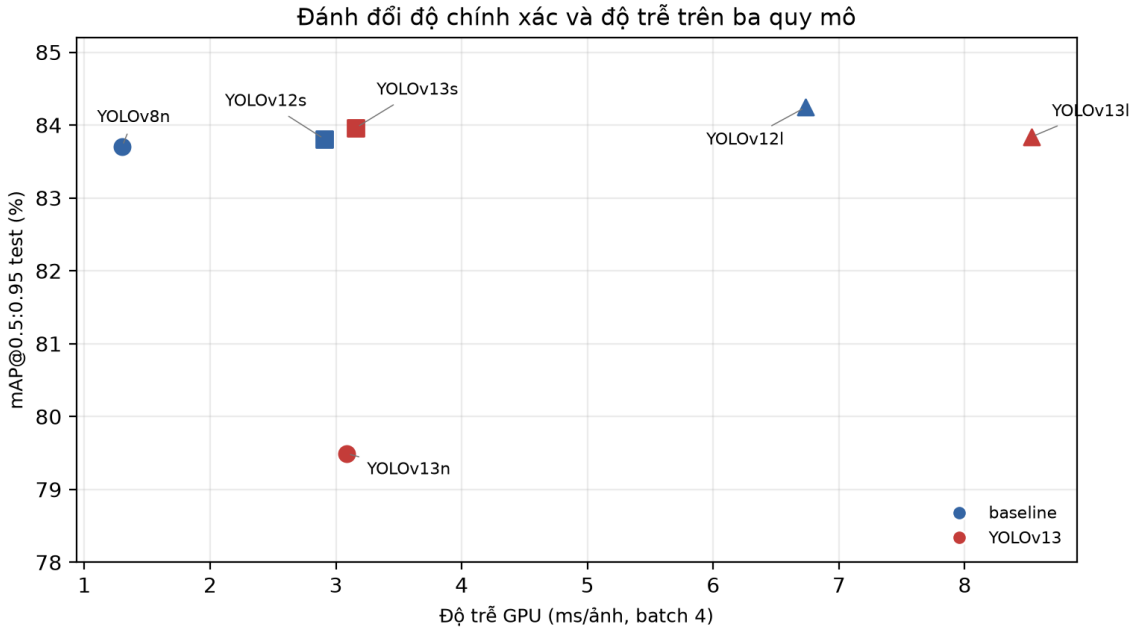
Mô hình	P (%)	R (%)	mAP@0.5 (%)	mAP@0.5:0.95 (%)	Params (M)	FLOPs (G)	CPU (ms)	CPU FPS*	GPU (ms)	GPU FPS*
YOLOv8n	98.18	99.10	99.47	83.70	3.01	3.46	11.87	84.27	1.30	769.73
YOLOv10n	95.77	98.65	99.32	83.02	2.71	3.55	13.68	73.09	2.01	497.88
YOLO11n	97.95	99.10	99.37	83.65	2.59	2.72	11.87	84.23	1.69	590.53
YOLOv12n	97.24	99.55	99.41	82.52	2.55	2.73	13.35	74.93	2.49	402.31
YOLOv13n	99.52	98.65	99.49	79.49	2.46	2.69	16.89	59.19	3.08	324.38
YOLOv8s	96.63	100.00	99.33	83.72	11.14	12.10	34.81	28.73	2.12	472.52
YOLOv10s	96.03	96.85	99.14	81.65	8.07	10.46	33.61	29.75	2.54	393.68
YOLO11s	99.10	98.65	99.47	82.18	9.43	9.10	30.70	32.57	2.34	427.76
YOLOv12s	97.76	98.41	99.41	83.81	9.22	9.07	33.61	29.75	2.91	343.73
YOLOv13s	98.21	99.10	99.45	83.96	9.55	9.15	38.95	25.67	3.16	316.55
YOLOv8l	98.20	98.10	99.42	83.94	43.63	69.88	165.67	6.04	6.42	155.88
YOLOv10l	96.42	97.06	99.04	82.68	25.77	53.74	140.35	7.13	6.30	158.81
YOLO11l	98.81	98.20	99.45	83.91	25.31	36.87	108.12	9.25	5.09	196.28
YOLOv12l	98.84	98.20	99.45	84.25	26.92	38.23	124.92	8.01	6.74	148.46
YOLOv13l	98.21	98.61	99.44	83.84	27.57	37.61	144.76	6.91	8.53	117.25

4.2. Đánh đổi độ chính xác và tốc độ

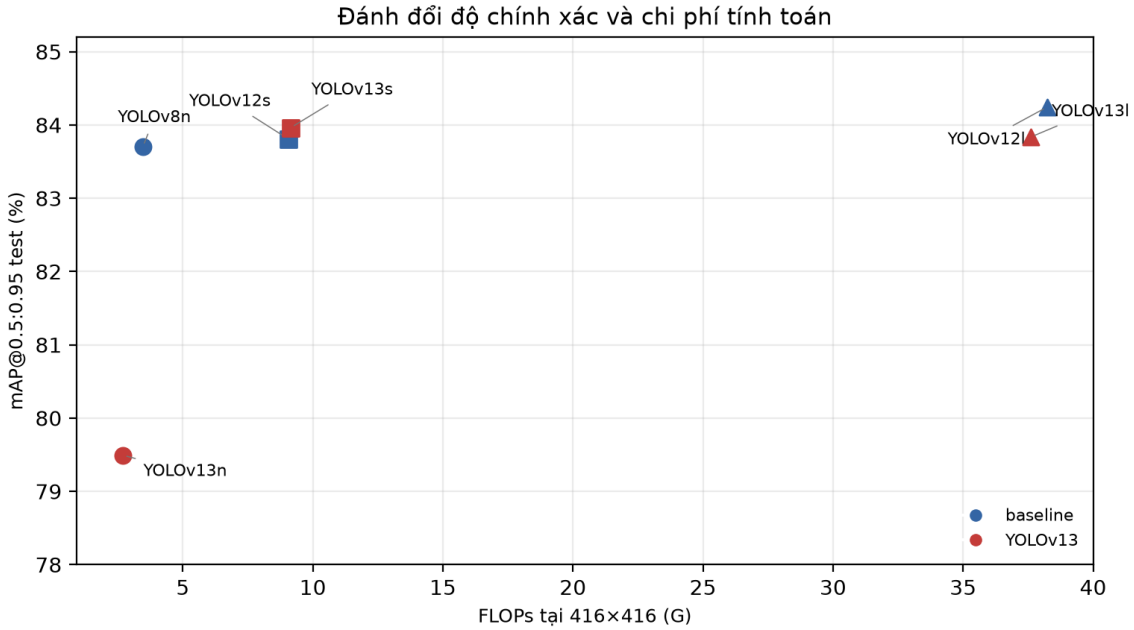
Bảng 2. YOLOv13 so với baseline cùng scale có validation mAP@0.5:0.95 cao nhất. Test chỉ được dùng sau khi chọn baseline.

Quy mô	Baseline tốt nhất theo validation	Val mAP95 (%)	Baseline test mAP95 (%)	YOLOv13	YOLOv13 val mAP95 (%)	YOLOv13 test mAP95 (%)	Δ test (điểm %)	Baseline GPU (ms)	YOLOv13 GPU (ms)	Δ latency (ms)
Nano	YOLOv8n	83.85	83.70	YOLOv13n	81.07	79.49	-4.21	1.30	3.08	+1.78
Small	YOLOv12s	84.78	83.81	YOLOv13s	84.72	83.96	+0.16	2.91	3.16	+0.25
Large	YOLOv12l	84.69	84.25	YOLOv13l	84.37	83.84	-0.41	6.74	8.53	+1.79

Ở Nano, YOLOv8n được chọn theo validation; YOLOv13n thấp hơn 4,21 điểm phần trăm mAP@0.5:0.95 trên test và có latency GPU cao hơn. Ở Small, YOLOv13s cao hơn YOLOv12s 0,16 điểm phần trăm nhưng chậm hơn 0,25 ms/ảnh (3,16 so với 2,91 ms). Ở Large, YOLOv13l thấp hơn YOLOv12l 0,41 điểm phần trăm và chậm hơn 1,79 ms/ảnh. Benchmark này không cho thấy YOLOv13 vượt trội đồng đều trên mọi scale.



Hình 1. mAP@0.5:0.95 và độ trễ GPU của baseline được chọn cùng YOLOv13



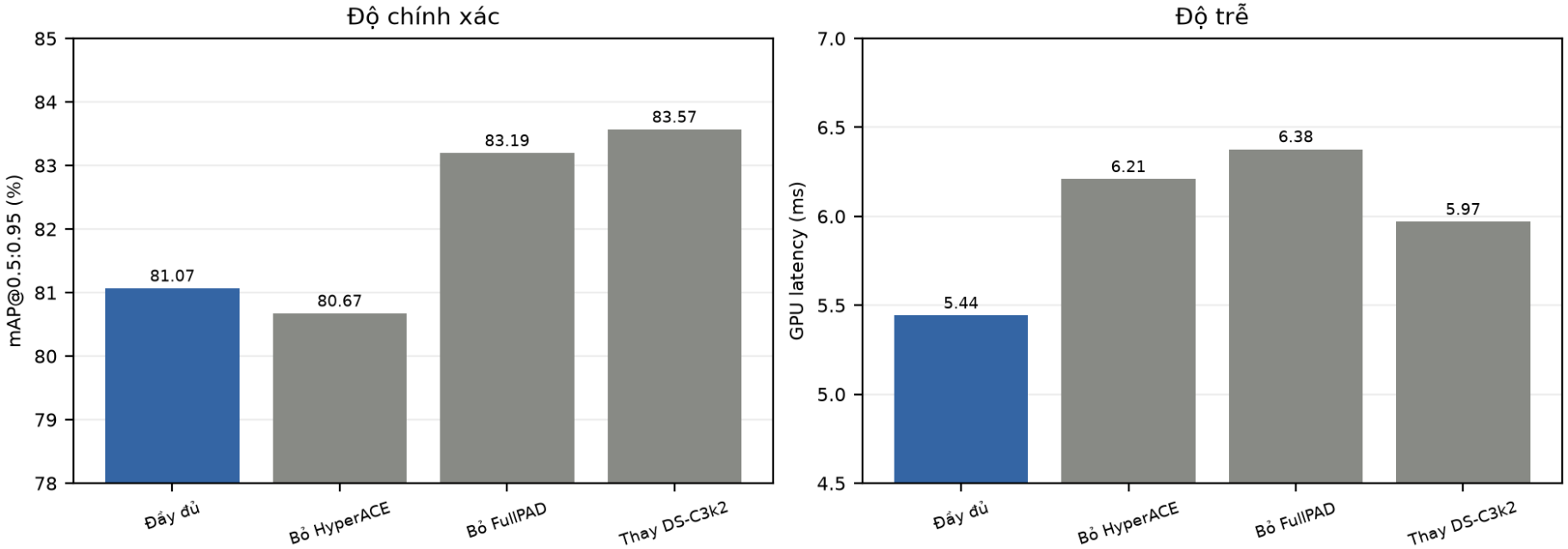
Hình 2. mAP@0.5:0.95 và FLOPs của baseline được chọn cùng YOLOv13

4.3. Ablation các thành phần kiến trúc

Ablation chạy trên YOLOv13-N với cùng pretrained initialization, split, seed, lịch và augmentation. Bảng dùng metric validation; Δ so với model đầy đủ. Biến thể no-DS-C3k2 thay thế 20 convolution mục tiêu.

Bảng 3. Ablation YOLOv13-N trên validation; latency đo trên GPU.

Biến thể	P (%)	R (%)	mAP@0.5 (%)	mAP@0.5:0.95 (%)	Params (M)	FLOPs (G)	GPU (ms)	Δ mAP95 (điểm %)	Δ Params (M)	Δ FLOPs (G)	Δ latency (ms)
YOLOv13-n-full	98.47	97.27	99.29	81.07	2.45	2.62	5.44	+0.00	+0.00	+0.00	+0.00
YOLOv13-n-no-hyperace	98.88	98.64	99.42	80.67	2.36	2.56	6.21	-0.39	-0.09	-0.06	+0.77
YOLOv13-n-no-fullpad	97.31	98.50	99.25	83.19	2.45	2.62	6.38	+2.13	+0.00	+0.00	+0.93
YOLOv13-n-no-dsc3k2	98.07	99.09	99.43	83.57	5.35	5.65	5.97	+2.50	+2.90	+3.03	+0.53



Hình 3. Độ chính xác và latency khi lần lượt loại bỏ hoặc thay thế thành phần

Bỏ HyperACE làm mAP@0.5:0.95 giảm 0,39 điểm phần trăm; latency đồng thời tăng 0,77 ms. Bỏ FullPAD làm mAP tăng 2,13 điểm nhưng latency tăng 0,93 ms. Thay DS-C3k2 làm mAP tăng 2,50 điểm, song tham số tăng 2,90 triệu, FLOPs tăng 3,03 G và latency tăng 0,53 ms. Vì vậy, ablation không chứng minh cả ba thành phần đều cải thiện đồng thời độ chính xác và tốc độ trên proxy. Một seed chỉ cung cấp bằng chứng mô tả, chưa hỗ trợ kết luận nhân quả phổ quát.

4.4. Đối chiếu với ablation đã công bố trên MS COCO

Bảng dưới đây tóm lược ablation trong bài YOLOv13. Đây là kết quả validation COCO của tác giả, không phải kết quả AdvT-shirt-1K. Đường phân phối đầy đủ đạt 48,0 AP@0.5:0.95, cao hơn cấu hình "no distribution" 0,9 điểm. DS block giảm tham số/FLOPs và gần như giữ AP; trên Small, AP@0.5 thấp hơn 0,1 điểm. Sự khác nhau giữa COCO và ablation cục bộ nhấn mạnh phụ thuộc dữ liệu, seed và cài đặt [30].

Thành phần / phép thử	Cấu hình	AP@0.5:0.95 (%)	AP@0.5 (%)	Params (M)	FLOPs (G)
FullPAD-HyperACE	Không phân phối (đối chứng trong bài)	47.1	64.1	NR	NR
FullPAD-HyperACE	Chỉ backbone → neck	47.8	65.0	NR	NR
FullPAD-HyperACE	Chỉ trong neck	47.6	64.8	NR	NR
FullPAD-HyperACE	Chỉ neck → head	47.7	64.9	NR	NR
FullPAD-HyperACE	Đủ ba nhánh FullPAD	48.0	65.2	NR	NR
DS block, YOLOv13-N	DS-C3k2	41.6	57.8	2.5	6.4
DS block, YOLOv13-N	Khối thường	41.6	57.9	3.1	7.9
DS block, YOLOv13-S	DS-C3k2	48.0	65.2	9.0	20.8
DS block, YOLOv13-S	Khối thường	48.0	65.3	11.7	27.1
Số hyperedge, YOLOv13-S	M=2	47.8	64.7	8.6	20.4
Số hyperedge, YOLOv13-S	M=4	47.9	65.0	8.8	20.5
Số hyperedge, YOLOv13-S	M=8	48.0	65.2	9.0	20.8
Số hyperedge, YOLOv13-S	M=16	48.1	65.2	9.6	21.5

Số liệu diễn giải phần văn bản và phép trừ từ Table V của bài gốc không hoàn toàn khớp cho DS block Small. Báo cáo dùng phép trừ trực tiếp từ bảng (tiết kiệm 2,7M tham số và 6,3 GFLOPs) và ghi nhận sai khác nguồn thay vì coi đó là phép đo mới [30].

5. Thảo luận

5.1. Trả lời câu hỏi nghiên cứu

RQ1 — Chất lượng định vị. YOLOv13s đạt 83,96% mAP@0.5:0.95 và 99,45% mAP@0.5 trên test. YOLOv12s là cấu hình cao nhất theo validation trên toàn benchmark, đạt 83,81% mAP@0.5:0.95 trên test. Chênh lệch giữa hai mô hình Small là +0,16 điểm phần trăm cho YOLOv13s trên test.

RQ2 — Chi phí suy luận. So với YOLOv12s, YOLOv13s có thêm 0,33 triệu tham số, 0,08 GFLOPs, và chậm hơn 0,25 ms/ảnh GPU, đổi lại mAP@0.5:0.95 cao hơn 0,16 điểm trên test. Đây là đánh đổi nhỏ ở scale Small, không phải cải thiện FPS. Kết quả không thay thế benchmark edge; cần đo trên thiết bị đích, với pipeline tiền/hậu xử lý, stream và công suất thực tế.

RQ3 — Khả năng phòng thủ. Thí nghiệm chỉ đánh giá định vị hộp patch. Không đo attack success rate, suy giảm recall của đối tượng, số proposal sau NMS, tải NMS hoặc khả năng khái quát hóa sang patch chưa thấy. Do đó, chưa có bằng chứng rằng YOLOv13 chống được tấn công vật lý hoặc ngăn DoS/NMS-overload.

5.2. Ý nghĩa các khối YOLOv13

HyperACE hướng đến liên kết nhiều vị trí đặc trưng cùng lúc thay vì chỉ quan hệ từng cặp; FullPAD truyền thông tin tăng cường qua các giai đoạn; DS-C3k2 nhắm giảm phép tính tích chập. Đây là lý do kiến trúc khiến YOLOv13 đáng khảo sát cho patch có tương quan giữa họa tiết, áo và người. Tuy nhiên, bằng chứng địa phương cho thấy bỏ HyperACE làm giảm nhẹ mAP, còn bỏ FullPAD hoặc thay DS-C3k2 lại tăng mAP trong cấu hình và seed này, đồng thời tăng latency hoặc chi phí. Không thể quy chênh lệch giữa các họ mô hình cho riêng một thành phần vì chúng còn khác backbone/head và pretrained weights.

Academic Discussion (English)

The local benchmark supports a narrow conclusion. YOLOv13-S records a small test mAP@0.5:0.95 advantage over YOLOv12-S in the unrounded run metrics, but is slower on the measured GPU; YOLOv13-N and YOLOv13-L fall below the validation-selected baselines. Results show a scale-dependent accuracy–latency trade-off, not uniform improvement. The high-order interactions represented by HyperACE are a plausible inductive bias for relating patch, garment, wearer, and scene context. FullPAD can redistribute enhanced features between stages, while DS-C3k2 is designed to reduce convolutional cost. These rationales describe the intended mechanisms; the one-seed local ablation does not establish that each component causes general accuracy or speed gains. The paper's COCO ablations are relevant architectural evidence from a separate dataset and protocol [30, 31].

All runtimes were measured on a desktop RTX 5060 or its CPU, at batch size four and through the local validator. They do not measure an edge board, camera pipeline, power, thermal throttling, NMS load under attack, or end-to-end frame deadlines. Edge suitability therefore remains an engineering hypothesis. A deployment study should export identical checkpoints to the target runtime, measure sustained streams, include decoding/preprocessing/NMS, record energy and missed deadlines, and report clean as well as adversarial performance. Recent patch-localization and defense studies also emphasize generalization to unseen patches and preservation of benign-image performance; neither is tested here [33–35].

Thảo luận học thuật (tiếng Việt)

Benchmark cục bộ chỉ ủng hộ một kết luận hẹp. YOLOv13s có mAP@0.5:0.95 test nhỉnh hơn YOLOv12s một mức nhỏ theo số liệu chưa làm tròn, nhưng chậm hơn trên GPU đã đo; YOLOv13n và YOLOv13l thấp hơn baseline được chọn bằng validation. Kết quả thể hiện đánh đổi độ chính xác–độ trễ phụ thuộc quy mô, không phải cải thiện đồng đều. Tương tác bậc cao mà HyperACE biểu diễn là một thiên kiến quy nạp hợp lý để liên hệ patch, áo, người mặc và ngữ cảnh. FullPAD có thể phân phối đặc trưng tăng cường giữa các giai đoạn, còn DS-C3k2 được thiết kế để giảm chi phí tích chập. Đây là mục tiêu kiến trúc; ablation một seed tại chỗ chưa chứng minh mỗi thành phần gây ra cải thiện tổng quát về độ chính xác hoặc tốc độ. Ablation COCO của bài gốc là bằng chứng kiến trúc liên quan nhưng thuộc dataset và protocol khác [30, 31].

Toàn bộ thời gian đo được lấy trên RTX 5060 máy bàn hoặc CPU của máy đó, batch bốn, qua validator cục bộ. Chưa có phép đo trên bo mạch edge, pipeline camera, điện năng, giảm xung nhiệt, tải NMS dưới tấn công hoặc deadline khung hình đầu-cuối. Vì thế, khả năng chạy edge vẫn là giả thuyết kỹ thuật. Nghiên cứu triển khai tiếp theo cần xuất cùng checkpoint sang runtime đích, đo luồng liên tục sau warm-up, tính cả giải mã/tiền xử lý/NMS, ghi điện năng và khung hình trễ, đồng thời báo cáo ảnh sạch lẫn ảnh đối kháng. Các nghiên cứu gần đây về phát hiện/phòng thủ patch cũng nhấn mạnh tổng quát hóa sang patch chưa thấy và bảo toàn hiệu năng ảnh sạch, hai điều chưa được thử ở đây [33–35].

5.3. Tính hợp lệ và giới hạn

- **Nội tại:** cùng split, ảnh vào, optimizer, augmentation, seed và validator tăng khả năng đối sánh; khác biệt pretrained weights và early stopping vẫn có thể ảnh hưởng thứ hạng.
- **Split:** chia theo ảnh trong thư mục nguồn, không theo người/patch/session; ảnh tương quan có thể rơi vào nhiều phân vùng.
- **Bên ngoài:** AdvT-shirt-1K là proxy định vị patch trên áo, không thay thế dataset SpongePatch/Physical-Visual DoS của đề tài.
- **Tốc độ:** latency/FPS phụ thuộc GPU, driver, runtime và batch; FPS* là nghịch đảo latency/ảnh, không phải throughput video.
- **Ablation:** một seed, chưa có khoảng tin cậy; cần chạy nhiều seed và test trên thiết kế/người chụp chưa thấy.
- **Triển khai:** chưa đo năng lượng, camera thật, xử lý đầu-cuối hay thiết bị edge chuyên dụng.

Công trình YOLOv13 công bố kết quả COCO để đánh giá kiến trúc. Các số COCO dùng dữ liệu, nhãn, thiết bị và giao thức khác nên không được ghép với kết quả local thành bảng xếp hạng trực tiếp. [30]

6. Kết luận và hướng phát triển

Conclusion (English)

YOLOv13-S achieved 83.96% test mAP@0.5:0.95 and 99.45% mAP@0.5 on this derived patch-localization benchmark. The run-level mAP difference from YOLOv12-S is about +0.16 percentage points before display rounding, while measured GPU latency is 0.25 ms higher. YOLOv13-N and -L did not surpass their validation-selected baselines. The local one-seed ablations and separate COCO ablations motivate further study but do not establish universal component-level gains. Because the original project splits have no images or labels, these findings apply only to the AdvT-shirt-1K proxy and do not demonstrate physical robustness, NMS-overload mitigation, or edge readiness. Follow-up work should use the original labeled data, group-disjoint splits, repeated seeds, direct attack/NMS measurements, and end-to-end profiling on the target device.

Kết luận và hướng phát triển

Nghiên cứu fine-tune và benchmark 15 cấu hình YOLOv8/10/11/12/13 Nano/Small/Large cho định vị patch trên split dẫn xuất AdvT-shirt-1K. YOLOv13s đạt 83,96% mAP@0.5:0.95 và 99,45% mAP@0.5; so với YOLOv12s, mức tăng là 0,16 điểm phần trăm mAP@0.5:0.95 cùng 0,25 ms latency GPU tăng thêm. Mô hình có validation mAP cao nhất toàn benchmark là YOLOv12s, với test mAP@0.5:0.95 83,81% và mAP@0.5 99,41%. Vì vậy, cặp số 83,81%/99,41% phải được gắn với YOLOv12s.

Kết luận chỉ áp dụng cho định vị patch trên proxy; thí nghiệm chưa xác lập tính kháng đối kháng, hiệu quả ngăn quá tải NMS hoặc lợi thế trên dữ liệu NCKH gốc. Bước tiếp theo là hoàn thiện nhãn đề tài, chia test độc lập theo người/thiết kế patch, lặp nhiều seed, đo attack success và tải NMS trực tiếp, sau đó đánh giá end-to-end trên phần cứng edge.

7. Tài liệu tham khảo

[1] T. B. Brown, D. Mané, A. Roy, M. Abadi, and J. Gilmer, “Adversarial Patch,” arXiv:1712.09665, 2017. https://arxiv.org/abs/1712.09665.

[2] A. Athalye, L. Engstrom, A. Ilyas, and K. Kwok, “Synthesizing Robust Adversarial Examples,” in Proc. ICML, 2018, pp. 284–293. https://proceedings.mlr.press/v80/athalye18b.html.

[3] I. Shumailov, Y. Zhao, D. Bates, N. Papernot, R. Mullins, and R. Anderson, “Sponge Examples: Energy-Latency Attacks on Neural Networks,” in Proc. IEEE EuroS&P, 2021, pp. 212–231. https://arxiv.org/abs/2006.03463.

[4] D. Wang, C. Li, S. Wen, Q.-L. Han, S. Nepal, X. Zhang, and Y. Xiang, “Daedalus: Breaking Non-Maximum Suppression in Object Detection via Adversarial Examples,” IEEE Transactions on Cybernetics, vol. 52, no. 8, pp. 7427–7440, 2022. https://arxiv.org/abs/1902.02067.

[5] A. Shapira, A. Zolfi, L. Demetrio, B. Biggio, and A. Shabtai, “Phantom Sponges: Exploiting Non-Maximum Suppression to Attack Deep Object Detectors,” in Proc. WACV, 2023, pp. 4571–4580. https://openaccess.thecvf.com/content/WACV2023/html/Shapira_Phantom_Sponges_Exploiting_Non-Maximum_Suppression_To_Attack_Deep_Object_Detectors_WACV_2023_paper.html.

[6] E.-C. Chen, P.-Y. Chen, I.-H. Chung, and C.-R. Lee, “Overload: Latency Attacks on Object Detection for Edge Devices,” in Proc. CVPR, 2024, pp. 24716–24725. https://openaccess.thecvf.com/content/CVPR2024/html/Chen_Overload_Latency_Attacks_on_Object_Detection_for_Edge_Devices_CVPR_2024_paper.html.

[7] C. Ma, N. Wang, Q. A. Chen, and C. Shen, “SlowTrack: Increasing the Latency of Camera-Based Perception in Autonomous Driving Using Adversarial Examples,” Proc. AAAI, vol. 38, no. 5, pp. 4062–4070, 2024, doi: 10.1609/aaai.v38i5.28200. https://ojs.aaai.org/index.php/AAAI/article/view/28200.

[8] T. Wang, Z. Wang, C. Wang, Y. Shu, R. Deng, P. Cheng, and J. Chen, “Can't Slow Me Down: Learning Robust and Hardware-Adaptive Object Detectors against Latency Attacks for Edge Devices,” in Proc. CVPR, 2025, pp. 19230–19240. https://openaccess.thecvf.com/content/CVPR2025/html/Wang_Cant_Slow_Me_Down_Learning_Robust_and_Hardware-Adaptive_Object_Detectors_CVPR_2025_paper.html.

[9] J.-P. Monteuuis, C. Chen, and J. Petit, “Latency NMS Attacks: Is It Real Life or Is It Just Fantasy?,” in Advances in Neural Information Processing Systems, vol. 38, 2025, doi: 10.52202/085713-1289. https://proceedings.neurips.cc/paper_files/paper/2025/hash/371713c3e5314dff9483c62c5abb98a8-Abstract-Conference.html.

[10] Ultralytics, “COCO128 Dataset,” tài liệu trực tuyến, truy cập ngày 02/10/2026. https://docs.ultralytics.com/datasets/detect/coco128/.

[11] J. Redmon and A. Farhadi, “YOLOv3: An Incremental Improvement,” arXiv:1804.02767, 2018. https://arxiv.org/abs/1804.02767.

[12] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in Proc. CVPR, 2018, pp. 4510–4520. https://openaccess.thecvf.com/content_cvpr_2018/html/Sandler_MobileNetV2_Inverted_Residuals_CVPR_2018_paper.html.

[13] Ultralytics, “Explore Ultralytics YOLOv8,” tài liệu phần mềm trực tuyến, truy cập ngày 02/10/2026. https://docs.ultralytics.com/models/yolov8/.

[14] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common Objects in Context,” in Proc. ECCV, 2014, pp. 740–755. https://www.microsoft.com/en-us/research/wp-content/uploads/2014/09/LinECCV14coco.pdf.

[15] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, “Intriguing Properties of Neural Networks,” arXiv:1312.6199, 2013. https://arxiv.org/abs/1312.6199.

[16] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and Harnessing Adversarial Examples,” arXiv:1412.6572, 2014. https://arxiv.org/abs/1412.6572.

[17] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards Deep Learning Models Resistant to Adversarial Attacks,” in Proc. ICLR, 2018. https://arxiv.org/abs/1706.06083.

[18] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” in Advances in Neural Information Processing Systems, vol. 28, 2015. https://arxiv.org/abs/1506.01497.

[19] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, “SSD: Single Shot MultiBox Detector,” in Proc. ECCV, 2016. https://arxiv.org/abs/1512.02325.

[20] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal Loss for Dense Object Detection,” in Proc. ICCV, 2017, pp. 2980–2988. https://arxiv.org/abs/1708.02002.

[21] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-End Object Detection with Transformers,” in Proc. ECCV, 2020. https://arxiv.org/abs/2005.12872.

[22] M. Alzantot, Y. Sharma, S. Chakraborty, H. Zhang, C.-J. Hsieh, and M. Srivastava, “GenAttack: Practical Black-box Attacks with Gradient-Free Optimization,” in Proc. GECCO, 2019, pp. 1111–1119, doi: 10.1145/3321707.3321749. https://doi.org/10.1145/3321707.3321749.

[23] D. Yang, Z. Yu, H. Yuan, and Y. Cui, “An Improved Genetic Algorithm and Its Application in Neural Network Adversarial Attack,” PLOS ONE, vol. 17, no. 5, e0267970, 2022, doi: 10.1371/journal.pone.0267970. https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0267970.

[24] A. E. Cinà, A. Demontis, B. Biggio, F. Roli, and M. Pelillo, “Energy-Latency Attacks via Sponge Poisoning,” Information Sciences, vol. 702, art. 121905, 2025, doi: 10.1016/j.ins.2025.121905. https://doi.org/10.1016/j.ins.2025.121905.

[25] J. te Lintelo, S. Koffas, and S. Picek, “The SkipSponge Attack: Sponge Weight Poisoning of Deep Neural Networks,” ITU Journal on Future and Evolving Technologies, vol. 6, no. 3, pp. 247–263, 2025, doi: 10.52953/XKBU4341. https://doi.org/10.52953/XKBU4341.

[26] S. Thys, W. Van Ranst, and T. Goedemé, “Fooling Automated Surveillance Cameras: Adversarial Patches to Attack Person Detection,” in Proc. CVPR Workshops, 2019. https://arxiv.org/abs/1904.08653.

[27] K. Eykholt et al., “Robust Physical-World Attacks on Deep Learning Visual Classification,” in Proc. CVPR, 2018, pp. 1625–1634. https://openaccess.thecvf.com/content_cvpr_2018/html/Eykholt_Robust_Physical-World_Attacks_CVPR_2018_paper.html.

[28] Z. Hu, S. Huang, X. Zhu, F. Sun, B. Zhang, and X. Hu, “Adversarial Texture for Fooling Person Detectors in the Physical World,” arXiv:2203.03373, 2022. https://arxiv.org/abs/2203.03373.

[29] T. Wu, L. Tong, and Y. Vorobeychik, “Defending Against Physically Realizable Attacks on Image Classification,” in Proc. ICLR, 2020. https://arxiv.org/abs/1909.09552.

[30] M. Lei et al., “YOLOv13: Real-Time Object Detection with Hypergraph-Enhanced Adaptive Visual Perception,” arXiv:2506.17733, 2025. https://arxiv.org/abs/2506.17733.

[31] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao, “Hypergraph Neural Networks,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 1, 2019, pp. 3558–3565. https://doi.org/10.1609/aaai.v33i01.33013558.

[32] W. Wang et al., “BadPatch: Diffusion-Based Generation of Physical Adversarial Patches,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2025, pp. 6303–6313. https://openaccess.thecvf.com/content/ICCV2025W/Findings/html/Wang_BadPatch_Diffusion-Based_Generation_of_Physical_Adversarial_Patches_ICCVW_2025_paper.html.

[33] S. Wu, J. Wang, J. Zhao, Y. Wang, and X. Liu, “NAPGuard: Towards Detecting Naturalistic Adversarial Patches,” in *Proceedings of CVPR*, 2024, pp. 24367–24376. https://openaccess.thecvf.com/content/CVPR2024/html/Wu_NAPGuard_Towards_Detecting_Naturalistic_Adversarial_Patches_CVPR_2024_paper.html.

[34] V. Kumar and A. Agarwal, “A Unified, Resilient, and Explainable Adversarial Patch Detector,” in *Proceedings of CVPR*, 2025, pp. 30387–30397. https://openaccess.thecvf.com/content/CVPR2025/html/Kumar_A_Unified_Resilient_and_Explainable_Adversarial_Patch_Detector_CVPR_2025_paper.html.

[35] I. Yankelev et al., “AntiStyler: Defending Object Detection Models Against Adversarial Patch Attacks Using Style Removal,” in *Proceedings of CVPR*, 2026, pp. 27936–27945.

https://openaccess.thecvf.com/content/CVPR2026/html/Yankelev_AntiStyler_Defending_Object_Detection_Models_Against_Adversarial_Patch_Attacks_Using_CVPR_2026_paper.html.

[36] W. Wang et al., “AdvT-shirt-1K,” authors’ project repository and dataset description. https://github.com/Wwangb/AdvT-shirt-1K.

[37] xingjunm, “Adv-T-shirt,” Hugging Face dataset card, license listed as Apache-2.0. https://huggingface.co/datasets/xingjunm/Adv-T-shirt.

[38] iMoonLab, “YOLOv13: official source code and model definitions.” https://github.com/iMoonLab/yolov13.

[39] A. Wang et al., “YOLOv10: Real-Time End-to-End Object Detection,” in *Advances in Neural Information Processing Systems*, 2024. https://arxiv.org/abs/2405.14458.

[40] Ultralytics, “YOLO11 Model Documentation and Software Citation,” software documentation; the project notes that no formal YOLO11 research paper has been published. https://docs.ultralytics.com/models/yolo11/.

[41] Y. Tian, Q. Ye, and D. Doermann, “YOLOv12: Attention-Centric Real-Time Object Detectors,” arXiv:2502.12524, 2025; see also the authors’ implementation. https://arxiv.org/abs/2502.12524; https://github.com/sunsmarterjie/yolov12.

Phạm vi dữ liệu và tính tái lập

Bản mirror có 1.123 ảnh và 1.548 hộp patch; nguồn tác giả nêu 1.131 ảnh [32, 36, 37]. Split đề tài gốc chưa có ảnh/nhãn nên toàn bộ điểm số trong bảng là trên proxy, không được trích như kết quả NCKH gốc. Ảnh, nhãn, YAML, dữ liệu tải về, Hugging Face cache, cấu hình YOLO, checkpoint, log và plot nằm dưới `project_yolov13/`; YAML proxy là `dataset/adv_tshirt_patch_detection.yaml`, còn YAML chuẩn giữ `path: ./dataset`. Cấu hình YOLO, runs và weights đều ở thư mục dự án. Số liệu có thể kiểm tra trong `results/adv_tshirt_patch_benchmark.csv`, `results/adv_tshirt_patch_ablation.csv`, `results/adv_tshirt_patch_dataset_manifest.json` và `results/external_dataset_sources.md`.